Aalto RSE training

This site contains the Aalto RSE training program. It's not a textbook or course, but a reference for use during mentoring and of what the expectations are - mainly of moving from an individual researcher to a consulting engineer in a team.

This site is under construction and "wiki rules" are in effect: if you see something, just make the update. Someone will come later and clean it up. I'm also happy if this is useful to others and there are suggestions to make it more useful outside Aalto. Current editor: rkdarst.

What is a RSE at ASC?

You can find plenty of definitions online: somewhere between researcher and software developer. Researcher whose output is measured by software (or data), not journal publications. And many more.

These are all true, but not really relevant for our training in ASC. You know how to be a researcher (and can learn), and know how to use the tools of software, data, and computing (and can learn). What we need to had is how to go from these to being a good collaborator between groups. If you were just doing these things, a group would hire you themselves (possibly with our help). What a new member needs to learn is how we operate as a team, how it's similar and different than typical research groups, and how we interact with research groups. It's a much less individual role than an academic researcher, with different types of skills.

What does "engineer" even mean? It's not just someone that knows some tools or practices. It's someone who can balance the limits of technology, the needs of the customer, and broader costs (to the customer and society. While a researcher can come up with ideas, and a developer can do the development they are asked to do, a RSE critically looks at the needs and comes up with the right solutions, taking into account the special needs and uncertainties of research.

You should always keep in mind that we are here to help research and researchers, not technology. This often means adapting the technology to what people actually need, not trying to force people to adapt to the technology. We should give confidence to our customers to carry on by themselves after we are done. We live and die by how our customers see us.

What makes Aalto RSEs different from academic researches:

- We focus on supporting other reserchears, instead of making our own research ideas. (We do come up with plenty of new ideas, but it's at the tool-layer, rather than the idea-to-publish layer).
- Our standards are higher. We are expected to know our stuff, and not just be learning it while we are doing. (this applies collectively, individually we are all learning new things)

What makes Aalto RSE different from RSEs in other places:

- We tend to have a wider variety of small projects going on and we have more emphasis on teaching and general support.
- We are a research infrastructure organizationally located in an academic school. We aren't "IT people", we are research support. Our mission is much wider.

Further reading

- Architects, engineers, and other infrastructure staff in universities by rkdarst.
- RSE lessons from Civil Engineering by rkdarst
- Aalto SciComp values and culture
- A not-so-brief history of Research Software Engineers
- History of Aalto RSE

General background information.

Roles intro

This section lists various roles and the training for them. The roles are not very formal and in practice everything gets blended together However, for the purposes of new people getting started and not having to learn everything all at once, it makes sense to try to classify them somehow.

Everyone will take their own path to get started. Teaching these roles is mainly (in 2026) via mentoring. Everyone make take their own path. Also note this material is a work in progress and will be developed along with the mentoring.

Role list

	Responsibilities
Project work	The default RSE job: for a project, learn and/or apply the necessary scientific and technical background and work on it over time in close collaboration with the customers. This includes project planning when projects are straightforward. Primary challenges include ongoing communication with the customers.
Garage support	Help users with drop-in questions in garage and other short-term support. Questions may be anything, and customers may be helped directly, you may call someone who can help, or they can be directed to other help (with advice). Primary challenges include the huge variety of questions which may come, and the need to think on your feet.

	Responsibilities
Teaching	Teaches (+other support) in courses, mainly using the CodeRefinery teaching strategies of co-teaching, collaborative notes, and livestreaming. Teaches with compassion and always reminds learners that stuff is actually difficult and people are not alone. Always maintains a critical mind about what should be taught (as opposed to blindly recommending best practices that aren't practical for the audienc).
Project planner (architect?)	This person has enough broad experience to to untangle what the customer actually needs even when they may not know (or it may not be possible). Meet with customers to create the initial project plan, critically ask questions, and guide the work. Mentors junior RSEs in the project process. Primary challenges include understanding the optimal solution when even the customer may not know what they need and conveying risks and limitations.
Technical mentor	Work with junior RSEs (or researchers) in the execution of projects, giving pointers and co-working to teach them new skills. Mentors junior RSEs in our project process. Primary challenges include the hands-on teaching and knowing the tech.
Unit liaison	Keep active communication with the unit, including telling them about general RSE activities, attending unit meetings, and keeping an active search for projects. Primary challenges include networking with diverse busy people.
Team Supervisor	Manage HR and financial practicalities, especially including ultimate responsibility for mentoring and productivity of all team members and the team itself.

Someone somewhat humorously described these roles by what tools peolpe use:

- Project work: code editor
- Garage: zoom
- Teaching: OBS and Twitch
- Project planner: email & docs
- Technical mentor: zoom or air (in-person sound transmission medium)
- Unit liaison: coffee room
- Team supervisor: spreadsheet

Project work

Summary

"Project work" refers to any bigger projects done for customers. Typically, a customer will request something, it's worked on over several weeks or months, and eventually it's concluded. There is often ongoing support and consultation afterwards. Projects may take the form of long-term consulting or co-working, or a whole aspect of a project being independently delegated to the RSE.

Initial planning is a major task, because far too often a project is started without a clear end goal. This is normal for research, but when things are being given to an outsider to work on, the expectations are higher. Often times, we have to help the customer figure out what they should want, but this starting planning is listed under Project planner.

Since the starting process is described under Project planner, the major challenge of this section is communication and co-working practicalities. How to record the time spent and effort? How to keep an open communication with the customers, so that the project doesn't go off the rails? And so on.

Main challenges/pitfalls

- Working within a research group, without actually being part of it.
- Maintaining close enough communication so that expectations are met.
- Fully understanding what's going on and the expectations.
- Realizing when the customer doesn't just need something done, but needs help coming up with how to do it. You are a researcher, not a contracted developer.
- Gracefully handle major changes to the plan, possibly throwing everything off track.
- Gracefully handle customers who have something else come up and thus stop being responsive to us at all. Figuring out what you should do then: how to avoid it looking like you were inactive when they come back (unless that is what was desired).
- Not recording or being clear when the project goals drift. Then ending up at a point where we and the customer have different expectations and are unhappy.
- Customers who don't know as much about the project as you do (managing their expectations, keeping them in the loop, etc.).

Expectations / checklists

- When the request is clear and you can do things without help, negotiate and start projects yourself. When it's complex or there is any chance that the customer needs you to help develop the idea, use the ideas from Project planner.
- Use a researcher's mind and a developer's tools to solve the customer's problems.
- Maintain active communication with the customers, especially making sure that we avoid common pitfalls (see above)
- Discuss frequently with your mentors to ensure that you stay on track and they have the info to intervene if it ever became needed.
- Keep the customers up to date with what you are doing (for example at least weekly but ideally more), in a language customers can understand, in enough detail so that they can warn you if something is going wrong.
- Use the rse-projects repository issue tracker to keep track of what is going on
- Record worktimes via Halli {if,as} needed - and know if it should be.
- Ask for help when you need it.
- Give a report in the RSE meeting once a project is done.

External materials

- Project management: RSE perspective
- Prioritization and reporting
- Many other things on Research Software Engineers
  - Checklists section

- Timetracking via Gitlab procedures

Training program: materials and exercises

Demonstration: going through the rse-projects gitlab tracker

It can be useful to set a calendar time each week to send a status report to each major project you have. This reminds you to do it and reserves the time.

Garage support

Summary

The "SciComp garage" is our daily office hour, where people can come and ask us anything. Answering these questions is a quite demanding responsibility, and you need to be able to think on your feet and not get stressed when you don't know something.

Garage is not just a time to help others, but the common time we can talk with each other, and most importantly learn from each others' answers. When you see someone give an answer you aren't familiar with, always take the time to ask "what did you just do?" so that you can learn more.

Main challenges/pitfalls

- You will often be asked things you don't know right away. (A broad solution is: ask for more info in the way you would debug. For example, start by asking to share the screen and show the exact problem).
- You need to be able to know other people (ASC/RSE/ITS/research services/etc.), to be able to refer customers to them when appropriate.
- A lot like the "planner" role, you need to be able to understand lots of different things and advise on the best solution trade-off for each customer individually, without the time to take a break to plan.
- Helping people while still giving them confidence that they can and should keep trying themselves (and not over-playing your confidence).
- "Too many cooks" problem when too many staff try to support the same person, simultaneously asking questions and giving instructions.

Expectations / checklists

- Attend the garage whenever it's convenient (usually we have plenty of people, so if something else comes up, do that instead). However, attend often enough to stay connected to the team.
- Treat all customers with the highest level of respect, regardless of their background or current experience level.
- Make sure someone greets all incoming customers.

- For each support case, make sure that one person (and not more than one) is the primary communicator, to prevent the customer from getting overloaded.
- Ensure that you understand the full scope of the customer's request before diving in, especially the questions listed on Help (has it ever worked? what are you trying to accomplish? what did you do? what do you need?) (Avoid the XY problem.)
- Recognize difficult communication situations and act to make it better.
- When someone departs garage, give them encouragement to keep trying themselves, that "yes this is hard, but you can do it".
- You may ask people to come back another day (or schedule another meeting) if you need more time to figure it out, or you need a different person to be present.
- Help as best you can and ask for help from others when needed.
- Observe other answers so you can learn new things, and ask follow-up questions once the customers leave.
- Record visits in the garage diary, but don't make a big deal out of gathering this information.

External materials

- How to help someone use a computer by Phil Agre
- How to ask for help with supercomputers by Radovan Bast
- Online work and support

Training program: materials and exercises

- How to actually respond to user support requests?

Roleplay: run through a support request.

Roleplay: Run though a support request where you don't know the answer

Roleplay: Run through a support request where you need to tell them you can't help them.

Roleplay: Garage, someone has only used AI and doesn't know what their code does

Roleplay: Discussing with another staff member about a sub-standard support session

Other

- Psychology of support * How to help someone use a computer * Roleplay: garage help * Exercise: comment on a simulated bad support session * Roleplay: suggest to a colleague that their support could have been improved * Finding the real problem within the question
- Usability
- **Teamwork**

    - Work together but one person in charge of each support session

Teaching

Summary

Aalto RSE spends a significant amount of time teaching, because that's a good way to help the widest number of people. It also helps to bring projects into us. When we see something that is taking up a lot of our support time, we can add it to our courses to save time long-term.

Teaching isn't just being in front of people and talking, but there are very many parts to our collaborative teaching: co-teaching, collaborative notes, livestream broadcasting, lesson development, organizing courses, and more.

Main challenges/pitfalls

- Getting use to the CodeRefinery teaching style (though once you are used to it, it's actually less work)
- Managing time for lesson maintenance and development.
- Teaching with the mindset of a learner (and what thy need to know), not trying to teach them to be like you, an experienced RSE.

Expectations / checklists

- Take part in the Instructor kickstart program.
- Optionally, take part in CodeRefinery "train the trainer"
- Begin by co-teaching working lessons with experienced instructors. Do this 1-2 times.
- Prepare before teaching: Teaching plan and Lesson review checklist.
- Set up you computer for teaching following all the best practices (in addition to all the other things you learn during the instructor kickstart).
- Take on the role of primary instructor during workshops.
- There are other things you can do as well:
    - Broadcasting
    - Video editing
    - Course organizing
    - Major lesson development
    - etc.

External materials

- Demo of livestream teaching (read the video description) by rkdarst
- Motivation to CodeRefinery instructor training by rkdarst (includes ideas for teaching).
- All CodeRefinery manual pages

Training program: materials and exercises

- Instructor introduction
- Instructor kickstart program
- CodeRefinery Train the Trainer

Other

- Course arrangement
- Teaching philosophy

Project planner

Summary

The “project planner” may be better called a project mentor, or architect, or something like that. (If you have a better name, let me know). This role takes a new project request from a customer, figures out what they actually need, and makes the plan for how to do it. If the task is very clear, one could go straight to doing it (Project work), but many times we are the computing expert, not the customers. In these cases, we need to work with the customer (domain expert) to figure out what is needed.

The “project planner” is distinct from Project work since you may hand off the main work to someone else to actually do, possibly serving as their Technical mentor during the project. In short, figuring out what to do is usually harder than doing it (at least it’s a different skill). That’s why we have multiple people involved.

Main challenges/pitfalls

- Customers may not know what they need (and maybe not even what to ask for). If we do exactly what they ask, we may do the wrong thing. We need to carefully work to find out what the goals are. This is research, and you are a part of a research team, not a contracted developer.
- The job requires having a broad knowledge of what tools are available and even going and learning more to plan the project.
- Defining what is our responsibility and what is the customer’s responsibility.
- Planning enough to know what to do, while taking into account that in research the long-term plan may not be known.
- Even after you do know what to do, explaining it clearly and concisely enough that the customers and others can understand what is happening. (This often happens when needing to present the project to more customers than the original ones.)
- All the risks that come with research and software development, combined.

Expectations / checklists

- Serve as the second contact to projects (first contact may be someone hearing the broad idea and suggesting a meeting) and organize a planning meeting with the right people.
- During the planning meeting, really figure out what the people need (which may require digging, and also helping to come up with what the right solution is). Assume you are part of a research team, not a contracted developer.
- Use your experience and communication skills to develop a workable plan along with the customers.
- Write down the plan so that everyone is on the same page and expectations are known across all sides. Clearly allocate the different types of work to RSEs or academics.
- Avoid the XY problem.
- Ensure that customers are aware of any risks and you have a plan if the risks come.
- Have initial finance discussions (team supervisor needs to finalize it).
- Present the project in the weekly RSE meeting and your evaluation if we should take it or not. During this meeting, also help the team come up with the right people to do the project. You don't necessarily have to do every project yourself, even if you are involved in planning them.
- Have the courage to say "no" when that is the best answer.
- If someone else new is working on the projects, mentor them with the material from Project work.
- If you aren't working directly with the project, stay engaged enough to keep the project on-track, either as a Technical mentor, domain mentor, or general mentor role.

External materials

- Project starting template
- Project management: RSE perspective
- Prioritization and reporting
- Project lifecycle
- For group leaders
- The Zen of Scientific computing
- ... and almost everything else on Research Software Engineers

Training program: materials and exercises

Roleplay: Planning meeting of a new RSE project, including filling the template, making a Gitlab issue, etc.

Technical mentor

Summary

The technical mentor isn't so much a role, but skills that are used in all the other roles. Mentoring is just so important that we give it a top-level entry. A top-level entry also ensures that people can get credit for mentoring and reserve time for it.

As of 2026, we think we do this fairly well. Garage provides informal time for internal mentoring, and external mentoring. We expect to hire people who are willing to speak up and ask for help (to pull in internal mentoring when they need it), but we need to be aware if this assumption ever breaks.

Main challenges/pitfalls

- Mentoring is hard and a different job from doing things yourself.
- Computing is our life, but only a side-project for many of our customers. We are happy to spend time experimenting and learning by doing, other customers may want a more direct solution. Customers are very good at other things.

Expectations / checklists

- Check Garage support for some relevant things.
- Treat all customers with the highest level of respect, regardless of their background or current experience level.
- Before diving into a session, make sure you explicitly discuss what the mentee wants and how they expect to get there.
- When done, ask if you have been helpful and have done what they needed.
- As a doctor, you learn to never ask patients 'did you understand me?' Instead, you always ask 'did I explain it clearly enough?'" That way, the burden of understanding is on you, not the patient. The same philosophy is useful for technical mentoring.
- If users apologize for not knowing things, reject it - say "it's OK, no one is born knowing this, everyone asks similar questions, etc. Learning is a long process, you've focused on other things. We are here to help."
- Plenty has been written about mentoring… I'm not going to write much more here right now.
- If a customer or mentee is ever frustrating you and you realize you are reaching your limits, recognize this. Ask for help or even hand it off.
- If you realize that someone is completely in over their head and needs more fundamental help, also realize this and let your team leader know.

Internal mentoring:

- Be aware of who is new on the team. Talk to them. Invite them to see what you do, when you do new things. Tell them enough so that they'll keep talking to you later.
- Give extra time to answer new people's questions, go and screenshare and so on.
- As the team gets larger, don't assume people will speak up as often as you did when the team was smaller.

External materials

- How to help someone use a computer by Phil Agre
- Video: support services vs diversity.
- XKCD #2501 (average familiarity)
- Carpenteries instructor training, Expertise and instruction

Training program: materials and exercises

- See Garage support - some of the materials and exercises there are relevant here and can sometime be split up.

Unit liaison

Summary

A unit liaison mains close connection with the various other units, presenting the work of ASC to them and often being the first line of starting a new project. They may be the Project planner of some of that unit's projects, but they probably aren't the only one doing that (and it isn't necessarily the expectation).

For example, we may have unit liaisons to CS, NBE, PHYS, FCAI, ELLIS, IT Services, etc.

Main challenges/pitfalls

- It helps to have a pre-existing connection to the unit.
- It can be difficult to remember take the time to reach out when you have so many other things going on - it's always a low priority, when you are already too busy with existing projects.
- It can be difficult to explain why we exist and why people should use our services (academics want to do things themselves).
- Customers expect you to have a broad knowledge of everything ASC does.
- All the typical things that are involved in being a customer relations person.

Expectations / checklists

- Be a first line of contact to the unit (which doesn't necessarily mean you should be getting all the personal email from them, but you can grab their stuff from other issue trackers, advise others when you see their issues, etc.).

- Be broadly aware of what is going on in your unit.
- Be the voice of that unit to our team.
- Stay in touch with the coordinators/leaders of the unit and get invitations to important events.
- Where practical, go to the unit meetings and other events to represent the team.
- Run Group meetings for groups within the unit, or at least help arrange them and get others to come.

External materials

- Group meetings
- Aalto RSE for researchers 2025

Training program: materials and exercises

This is probably mentored and learned on-the-job.

Team supervisor

Summary

The team supervisor is the one official responsible for the team. They manage the HR, finance, and organizational aspects. They are ultimately responsible for making sure that every other part of the team works well.

Main challenges/pitfalls

- It's a different job that any of the other jobs.
- Aalto provides some training, but you still need to learn most things as you go.

Expectations / checklists

- Ensure the team mentoring and onboarding system is working well, and all team members are happy and have what they need.
- Maintain contact with HR and Finance
- Decide about matters for project funding.
- Be aware of the team enough to know if there are problems, and deal with them.
- Intervene in projects for which the customer is not satisfied.
- Coordinate hiring.

External materials

- The Manager's Path: A Guide for Tech Leaders Navigating Growth and Change (book, available online from the Aalto library).

Training program: materials and exercises

Mostly mentored and learned on-the-job.
This section tells what to expect at Aalto. It's not expected to be read all at once when starting, but slowly mentored according to each person's onboarding plan.

IT in Aalto

- Arrangement of ITS
- Data storage
- Compute services
- Department IT
  - Who is there
  - Services
  - CSIT
  - PHYS IT
  -
- CSC * HPC * Data services * Server computing services * Teaching

Research services at Aalto

Tech stuff

This list is made for new people, who have been hired as RSEs at Aalto. This isn't a list of what someone has to know to apply, **and not what people should know before starting**. It only provides a map, a RSE will incrementally learn things here (and probably things not on the list - this list is what we already know). In the future, we expect this list to be copied and reused in other contexts.

Someone might take ~6 months to slowly learn things on this list as they need them. Not everything will be needed.

Linux and shell

- Shell scripting and the OS interface
  - Advanced Bash SCripting guide: https://tldp.org/LDP/abs/html/
- Containers
  - Docker: https://docs.docker.com/get-started/overview/ (though we don't Docker on the cluster, it's good to know anyway)
  - Dockerfile reference: https://docs.docker.com/engine/reference/builder/
  - Apptainer (formerly called "singularity"):
    - This is what is actually used on the cluster
    - https://apptainer.org/docs/user/latest/

    - Key things: make own container, convert docker to apptainer, how to run on cluster
    - singularity_wrapper and how it works with Lmod: https://scicomp.aalto.fi/triton/usage/singularity/ (load a singularity module and you can see where the script is)
- Lmod:
  - https://lmod.readthedocs.io/en/latest/
  - (hint: personal modulefiles: mkdir ~/modulefiles ; module use ~/modulefiles)
  - Mainly basic use + writing modulefiles
- Conda
  - especially resolving GPU code related issues
-

Software development tools

- CodeRefinery lessons (https://coderefinery.org)
- git-intro: https://coderefinery.github.io/git-intro/
- and git-collaborative: https://coderefinery.github.io/git-collaborative/
- Reproducible Research: https://coderefinery.github.io/reproducible-research/
- Documentation: https://coderefinery.github.io/documentation/
- (Jupyter: https://coderefinery.github.io/jupyter/)
- Automated Testing https://coderefinery.github.io/testing/
- Modular type-along or Modular code developmenent presentation
- Social coding: https://coderefinery.github.io/social-coding/

There are a few other interesting CodeRefinery lessons: https://coderefinery.org/lessons/

HPC

- Triton tutorials is what we expect our users to know, and reading through these is enough (it will be familiar): https://scicomp.aalto.fi/triton/#tutorials
- And in general, browse (but not read in detail) the rest of the Triton https://scicomp.aalto.fi/triton/

Programming

Python

- Python virtual environments and Conda environments: https://scicomp.aalto.fi/scicomp/python/ , https://scicomp.aalto.fi/triton/apps/python/
  - Be able to create a virtual environment
- Python module/package structure

- Python packaging
  - https://packaging.python.org/en/latest/tutorials/packaging-projects/
  - setup.py vs pyproject.toml (newer)
- Python command line interfaces (argparse), installing interfaces via packages, ...
- Other steps for a good project
  - Good project structure (module-name/module_name/)
  - Command line interface
  - Modular and maintainable code
  - Installable: setup.py vs pyproject.toml
  - Linter (if worth it)
  - Test coverage (if worth it)
  - Good documentation (README, code-level docs, sphinx + RTD/gh-pages)
  - Automated tests to the degree useful for the project. At least minimal.
  - Github Actions
  - PyPI release
  - conda-forge release
  - GH-action for releasing to PyPI/conda-forge

Examples: *

Data processing

- webdataset
- Small file management in various ways
- Exercise: i/o benchmarking

Data management

- FAIR data
- Open Science
- Aalto Data Agents webinars

Web stuff

- intro, debugging
- django?

Data management

- Lists key open science events: https://openresearchcalendar.org/calendar/
- https://avointiede.fi/en
  - The Finnish Open Science Days are good, too, but most content is in Finnish.

Ethics

- Ethics of science
  - Reproducibility
  - Academic credit
- Personal data
  - Definitions
  - Legal bases of handling
  - Required steps
  - How Aalto process works
- AI ethics

Security

- Basics
  - Don't share accounts, know your data rating, handle it correctly
- Know basics
  - Decide security level needed for data. Don't over classify
  - Select right systems for this
  -
- Common working practices

(section is not yet filled out.) This section is a collection of interesting links and things to learn as RSEs. It's also not intended to be followed one-by-one but a general reference for what you may need to know, that is easy to be missed in academic training. Your mentor will tell you what to focus on.

12 session training plan

This is a draft if we wanted to make an actual course out of becoming a new RSE with ASC. This will likely never happen, since we are all starting at different times and so far it's been better to mentor people as they come in. Still, it's a possibility.

- user support (garage, issue tracker, esupport, ASC standards for support)
- Storage systems (tiriton scratch, netapp,)
- project data arrangement (arranging files in an individual project. arranging group project dirs.)
- Ethics (personal data, sanctions etc., need for reproducibility)
- Security (data classification, storage&processing systems, common procedures.)
- Triton advanced (slurm internals, architecture, parallelism)
- Conda (basic use, common problems)
- Software installation (self via pip&conda&R, scibuilder-conda, scibuilder-spack)
- Python software (environments, packaging, command lines. Exercise: make a simple package, install from Github., install from github)
- R software (see above for Python)

- Git and Github (basics should be known. multi-person projects, issues, PRs, organizations, advanced features.)
- Data storage performance (small file optimization, local disks. Exercise: benchmark different filesystems)

Other RSE training

Other links to things that look like RSE-type training, or could be used as RSE training programs.

- For RSE candidates and community
- https://rsetoolkit.github.io/
- https://society-rse.org/resources/
- https://www.hpc-certification.org/
- https://eumaster4hpc.eu/

Other links and ideas not yet sorted to the above sections.